

Método Black - Litterman

El método Black-Litterman es una técnica de asignación de activos que combina la teoría del portafolio de Markowitz con las opiniones subjetivas de los inversionistas. Fue desarrollado por Fisher Black y Robert Litterman en 1992. Este método se utiliza para generar una asignación de activos óptima teniendo en cuenta las creencias o perspectivas del inversionista sobre los rendimientos esperados de los activos. A diferencia del enfoque tradicional de Markowitz, que utiliza únicamente la matriz de covarianza de los rendimientos y los rendimientos esperados, el método Black-Litterman incorpora información adicional sobre las perspectivas y creencias del inversionista.

El proceso se puede resumir de la siguiente manera:

1. Estimación de los rendimientos esperados: Se comienza con una estimación inicial de los rendimientos esperados de los activos. Estos pueden ser basados en análisis fundamental, datos históricos u otras fuentes.
2. Especificación de las creencias del inversionista: El inversionista proporciona sus creencias subjetivas sobre los rendimientos esperados de los activos. Estas creencias se expresan en forma de un vector de perspectivas de rendimientos.
3. Incorporación de las creencias del inversionista: Las creencias del inversionista se combinan con la matriz de covarianza de los activos para obtener una matriz de covarianza ajustada. Esto se realiza mediante el Teorema de Bayes, que permite actualizar las estimaciones iniciales de los rendimientos esperados en función de las creencias del inversionista.
4. Optimización de la asignación de activos: Utilizando la matriz de covarianza ajustada y los rendimientos esperados actualizados, se realiza la optimización de la asignación de activos para determinar los pesos óptimos de la cartera.

El método Black-Litterman es especialmente útil cuando se desea incorporar las creencias subjetivas del inversionista en el proceso de toma de decisiones. Permite combinar las perspectivas del inversionista con la información disponible en el mercado para generar una asignación de activos más robusta y personalizada.

Implementación en Python

1. Se definen los datos iniciales de la cartera, incluyendo la matriz de covarianza de los activos, los pesos de mercado, la aversión al riesgo del inversionista y los rendimientos esperados iniciales.
2. Se calcula el rendimiento de mercado multiplicando los pesos de mercado por los rendimientos esperados iniciales.
3. Se ajusta la matriz de covarianza mediante la aplicación del Teorema de Bayes, utilizando la matriz de confianza de las perspectivas y la matriz de covarianza original.
4. Se calcula el factor de ajuste multiplicando la inversa de la matriz de covarianza ajustada por el producto de la matriz de confianza de las perspectivas y el vector de perspectivas de rendimientos.
5. Se calculan los nuevos rendimientos esperados utilizando la matriz de covarianza ajustada, los pesos de mercado y el factor de ajuste.
6. Se define la función objetivo y las restricciones para la optimización de la asignación de activos. La función objetivo busca maximizar el rendimiento esperado de la cartera, mientras que la restricción asegura que la suma de los pesos sea igual a 1.
7. Se realiza la optimización de la asignación de activos utilizando la función `minimize` de `scipy.optimize`. Se utilizan los rendimientos esperados ajustados y se especifican los límites y restricciones de los pesos.
8. Se obtienen los pesos óptimos de la cartera.

In [11]:

```
# Modelos de Black - Litterman

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# Datos iniciales
cov_matrix = np.array([[0.04, 0.02, 0.01, 0.03],
                        [0.02, 0.09, 0.05, 0.01],
                        [0.01, 0.05, 0.06, 0.02],
                        [0.03, 0.01, 0.02, 0.07]]) # Matriz de covarianza de Los activos
market_weights = np.array([0.25, 0.3, 0.15, 0.3]) # Pesos de mercado de Los activos
risk_aversion = 0.05 # Aversión al riesgo del inversionista

# Estimación inicial de Los rendimientos esperados
expected_returns = np.array([0.08, 0.10, 0.06, 0.09])

# Matriz de confianza de Las perspectivas
confidence_matrix = np.array([[0.8, 0.0, 0.0, 0.0],
                              [0.0, 0.7, 0.0, 0.0],
                              [0.0, 0.0, 0.6, 0.0],
                              [0.0, 0.0, 0.0, 0.5]])

# Vector de perspectivas de rendimientos
views = np.array([0.02, 0.04, -0.01, 0.03])

# Cálculo del rendimiento de mercado
market_return = np.dot(market_weights, expected_returns)

# Cálculo del factor de ajuste utilizando el teorema de Bayes
posterior_covariance = np.linalg.inv(np.linalg.inv(cov_matrix) + np.dot(np.dot(np.transpose(market_weights), confidence_matrix), market_weights))
posterior_mean = np.dot(posterior_covariance, (np.dot(np.linalg.inv(cov_matrix), expected_returns) + np.dot(np.dot(np.transpose(market_weights), confidence_matrix), market_weights)))

# Función objetivo para La optimización
def objective(weights):
    return -np.dot(weights, posterior_mean)

# Restricciones
def constraint1(weights):
    return np.sum(weights) - 1.0

def constraint2(weights):
    return min(weights) - 0.10

# Optimización de La cartera
bounds = [(0, None)] * len(expected_returns) # Límites de Los pesos (0, infinito)
constraints = [{ 'type': 'eq', 'fun': constraint1 }, { 'type': 'ineq', 'fun': constraint2 }]
initial_guess = np.ones(len(expected_returns)) / len(expected_returns) # Peso inicial i
result = minimize(objective, initial_guess, method='SLSQP', bounds=bounds, constraints=constraints)
opt_weights = result.x

# Resultados
print("Rendimientos esperados originales:", expected_returns)
print("Rendimiento de mercado:", market_return)
print("Nuevos rendimientos esperados posteriores:", posterior_mean)
print("Pesos óptimos:", opt_weights)

# Gráfico de Los rendimientos esperados originales y nuevos
x = np.arange(len(expected_returns))
```

```
plt.bar(x, expected_returns, width=0.4, align='center', label='Rendimientos Esperados Or
plt.bar(x+0.4, posterior_mean, width=0.4, align='center', label='Rendimientos Esperados
plt.xticks(x+0.2, ['Activo 1', 'Activo 2', 'Activo 3', 'Activo 4'])
plt.xlabel('Activos')
plt.ylabel('Rendimientos Esperados')
plt.title('Comparación de Rendimientos Esperados')
plt.legend()
plt.show()

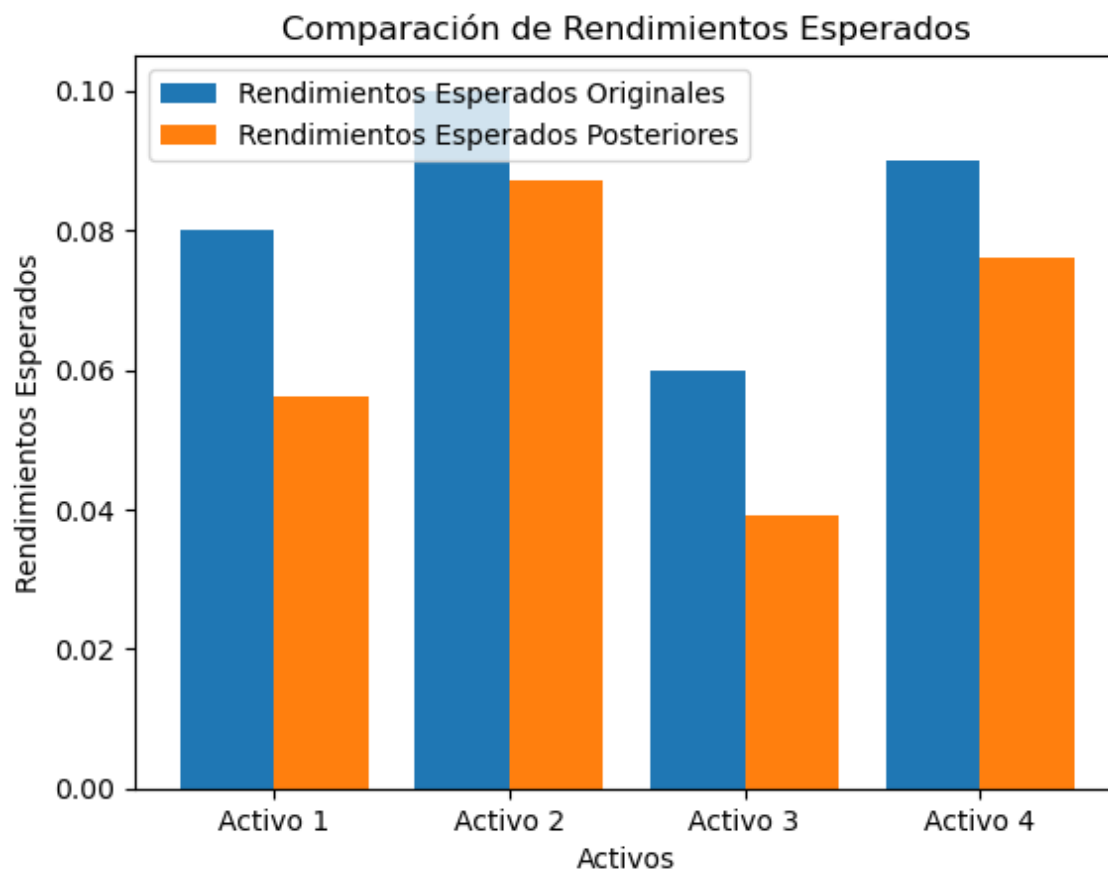
# Gráfico de Los pesos óptimos
plt.bar(x, opt_weights, width=0.4, align='center')
plt.xticks(x, ['Activo 1', 'Activo 2', 'Activo 3', 'Activo 4'])
plt.xlabel('Activos')
plt.ylabel('Pesos Óptimos')
plt.title('Pesos Óptimos de la Cartera')
plt.show()
```

Rendimientos esperados originales: [0.08 0.1 0.06 0.09]

Rendimiento de mercado: 0.086

Nuevos rendimientos esperados posteriores: [0.05624842 0.08702625 0.03914071 0.0759307]

Pesos óptimos: [0.1 0.69998602 0.09999977 0.10001421]



Pesos Óptimos de la Cartera

